Hand Gesture Recognition Using an MPU-6050 IMU, MSP430F5529, and Random Forest Classification

PHYS 319 -- Final Project Report

Nam Bui

April 2026

Abstract

This project builds a real-time hand gesture recognition system using an MPU-6050 IMU, an MSP430F5529 LaunchPad, and a machine learning pipeline on a personal laptop. The MSP430 reads motion data from IMU (accelerometer + gyroscope) over I2C at 10 Hz and sends it through UART to the laptop. A Python pipeline collects and processes every 20 rows of raw data, labels them with a gesture name, and trains a Random Forest classifier over three gesture classes: idle, swipe left, and swipe right. The trained model runs in real time and drives some applications: a real-time gesture display, a gesture password system, and a gesture-controlled terminal game. The system worked reasonably well after resolving several bugs related to UART baud settings, I2C communication, serial buffer contamination, dataset, and real-time state machine design.

1. Introduction

The long-term goal of this project is to build a low-cost personal coaching tool for badminton. An IMU worn on the wrist can capture swing motion data for machine learning models, without a camera (expensive) or human observer. If the system can classify different swing gestures reliably, it could eventually provide real-time feedback on technique and consistency. This project is the first step toward that goal: demonstrating that basic hand gestures can be recognized from raw IMU data in real time. Specifically, I investigated whether an MSP430F5529 microcontroller, an MPU-

6050 IMU, and a simple machine learning pipeline can reliably distinguish three gestures: idle, swipe left, and swipe right.

There are three stages: (1) MSP430 firmware reads the IMU and streams data over UART; (2) a Python pipeline collects training data, extracts features, and trains a classifier; and (3) real-time prediction scripts drive interactive applications.

2. Theory

2.1 Accelerometer and Gyroscope

An accelerometer measures how fast the device is accelerating. Inside the chip, a tiny mass is held by microscale springs. When the device moves, the mass shifts slightly, and the chip measures that shift as acceleration. One important side effect is that gravity always pulls on the mass, so even a completely still sensor reports a non-zero reading on the vertical axis. This means the raw accelerometer data always contains a gravity component that depends on how the sensor is tilted.

A gyroscope measures how fast the device is rotating. When the sensor is rotated, the mass inside spins and experiences a centrifugal Coriolis force, and the chip converts that force into an angular velocity reading.

In this project, MSP reads IMU data at 10Hz, which contains 6 features: ax, ay, az, gx, gy, and gz. From that, “a” represents the translational acceleration of the sensor along the corresponding axis, and “g” represents the angular velocity along the corresponding axis.

2.2 Feature Extraction

Since 6 features are read at a 10 Hz rate (1 row of data every 0.1 second), we cannot add a gesture label to every row of them (a supervised machine learning model needs a label for every row to train). This is because a gesture requires 1-3 seconds to perform.

The solution is to look at a window of 20 consecutive rows (2 seconds at 10 Hz) instead of one row at a time. From each window, seven new features are calculated for each of the six original sensor features: mean, standard deviation, minimum, maximum, range, absolute peak, and energy. This gives 7 × 6 = 42 features per window (each window gives one row of 42 features with one gesture label). These features capture the overall shape and intensity of the gesture motion while averaging out the noise. In real-time mode, the window slides forward by one sample at a time, producing a new prediction every 0.1 seconds.

2.3 Random Forest Classification

A Random Forest is a classifier that builds many simple decision trees, and the prediction is based on the majority vote of them. For example, there are 100 trees in this project, and if 70 trees vote “swipe right”, 15 trees vote “swipe left”, and 15 trees vote “idle”, the prediction is “swipe right”. A decision tree is a supervised machine learning model where a series of yes/no questions about the data leads to a final answer. For example: "Is the gyroscope range above 500? If yes, is the mean acceleration negative? If yes, the prediction is 'swipe left. ' Each tree learns from a slightly different random portion of the training data, so the trees make different kinds

of errors, and the combined result is more reliable than a single tree. Random Forest was chosen because it works well with small datasets, rarely overfits, and provides a confidence score for each prediction, which is useful for ignoring uncertain results. A neural network was considered but rejected because it requires significantly more data and is harder to tune for a simple three-class problem.

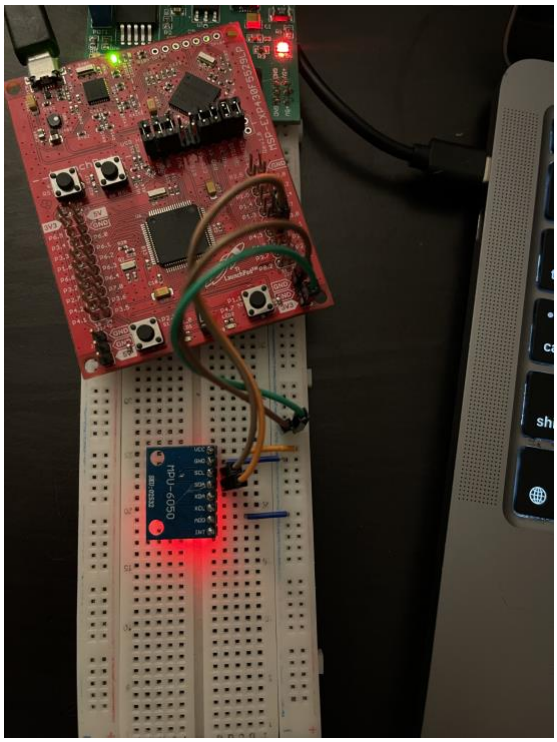
3. Apparatus

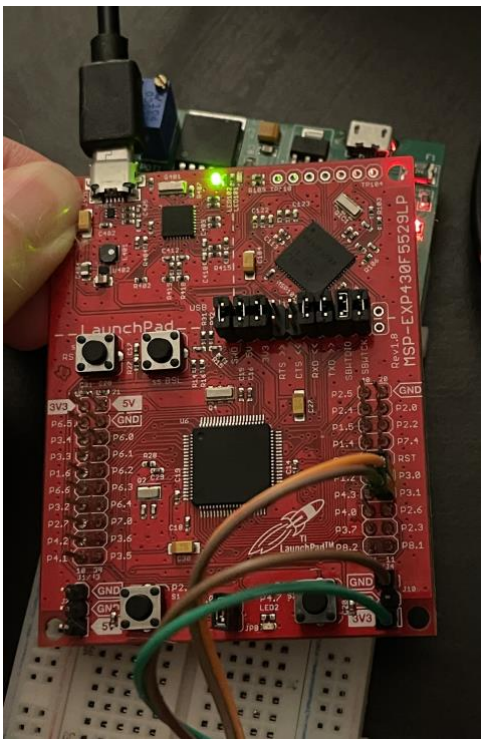
3.1 Hardware

The hardware consists of an MSP430F5529 LaunchPad and an MPU-6050 IMU breakout on a breadboard, connected by jumper wires. The MSP430 runs at approximately 1.048576 MHz using its internal DCOCLKDIV clock. The MPU-6050 provides three-axis accelerometer (ax, ay, az) and three-axis gyroscope (gx, gy, gz) readings as raw 16-bit integers over I2C.

3.2 Connections

The MPU-6050 is powered from the LaunchPad's 3.3 V rail. I2C uses peripheral UCB0: SDA on P3.0 and SCL on P3.1. The AD0 pin is tied to GND, setting the I2C address to 0x68. UART uses peripheral UCA1: TXD on P4.4 and RXD on P4.5, routed through the LaunchPad USB backchannel to the laptop (/dev/cu.usbmodem1203). Baud rate is 9600. The UART divider is BR0 = 109, UCBRSx = 2, matched to the 1,048,576 Hz SMCLK. The I2C clock divider is UCB0BR0 = 10.





Figures 1a, 1b: Wirings and connection between the MPU-6050, MSP430F5529, and laptop.

Signal	**MPU-6050 Pin**	**MSP430F5529 Pin**
Power	VCC	3.3 V
Ground	GND	GND
I2C Data	SDA	P3.0 (UCB0SDA)
I2C Clock	SCL	P3.1 (UCB0SCL)

Table 1: Pin connections between MPU-6050 and MSP430F5529.

3.3 System Pipeline

The full data flow is: MPU-6050 -> I2C -> MSP430 -> UART/USB -> Python -> ML model -> application output. The MSP430 firmware (main.c) reads all six sensor channels from the MPU-6050 as mentioned before.

In the laptop, five Python scripts handle different stages. “data_storing.py” collects and processes my own gesture data and saves it to CSV files for model training. “train.py” loads those files, extracts features, and trains the classifier. “predict1.py” runs live gesture prediction (new data). “password.py” and “game.py” implement the interactive applications. “inspect_data.py” is for checking and cleaning the dataset. Please see Appendix A and B for the description of the Python files and the firmware.

4. Results

4.1 Dataset

Data was collected across multiple sessions. The final dataset covers three gesture classes:

Class	**Raw Rows**	**Training Windows**
idle	1,620	81
swipe_left	1,480	74
swipe_right	1,500	75

Total	4,600	~230

Table 2: Dataset summary.

4.2 Classifier Performance

The Random Forest (100 trees) was evaluated with five-fold cross-validation. Feature importance analysis showed that gyroscope's mean along z axis and accelerometer's standard deviation along x axis features ranked highest – varying them will affect the prediction most. This is consistent with the expectation that wrist rotation during a swipe produces a distinctive angular velocity, for example, swiping left and swiping right yield positive and negative mean angular velocity along z axis respectively. Accelerometer mean along x axis ranks second highest, reflecting the fact that the 3 gestures' motion are significantly distinctive along x axis.

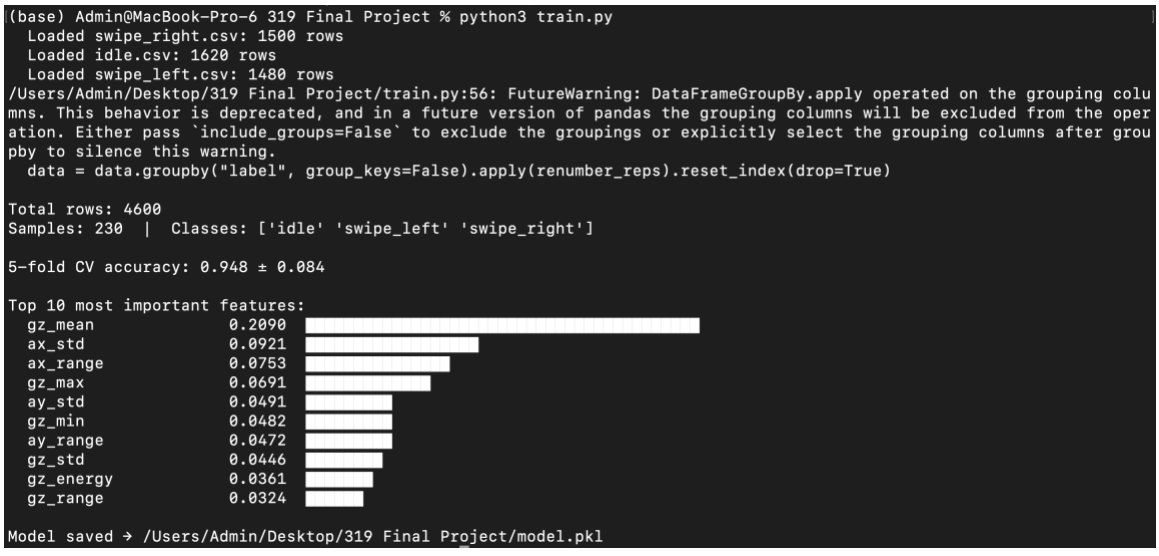


Figure 2: Bar chart of top 10 feature importance from the trained model and 5-fold cross-validation accuracy: 0.948 +/- 0.084 from train.py output.

4.3 Real-Time Behavior

In live use, the system effectively identified idle, swipe left, and swipe right. A clear swipe produced a confident prediction within one to two window updates (~1 second). Low-confidence predictions (below 60%) were suppressed. Both the password application and the dodge game responded correctly to gestures in normal use.

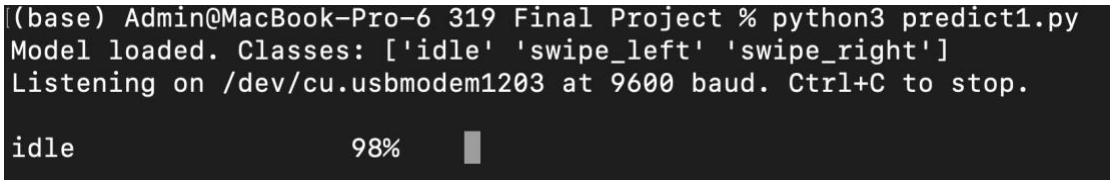


Figure 3: Real-time gesture prediction when running predict1.py. The confidence level is 98%, indicating 98/100 trees predict current gesture is "idle".

4.4 Debugging

Four bugs were identified and fixed during the project development.

4.4.1 UART Baud Mismatch

No serial output appeared, or the output was garbled. There were three causes: a typo in the Python port name, a baud rate calculated for exactly 1 MHz while the actual clock runs at 1,048,576 Hz, and a firmware bug where setting only one clock field accidentally switched the other clocks to 32 kHz. The port name was corrected, the baud divider was updated to BR0 = 109 to match the real clock, and the clock setup was rewritten to set all fields explicitly. The firmware was also updated to print "UART OK" on startup to confirm communication before anything else runs.

4.4.2 I2C NACK and Firmware Hang

The MPU-6050 did not respond, and the firmware froze while waiting for a reply that never came. The SDA and SCL wires were swapped, and the code had no way to handle a non-response, so it looped forever. The wiring was corrected (SDA to P3.0, SCL to P3.1, AD0 to GND), and the I2C code was updated to detect a NACK and return an error instead of hanging. The firmware now prints “MPU6050 OK” or “MPU6050 NACK” on startup, so wiring problems are immediately visible.

4.4.3 Serial Buffer Contamination

While running data_storing.py, the collection script was saving old data instead of fresh gesture data. Because the MSP430 streams continuously, the laptop’s serial buffer filled up with idle readings while waiting for the user to press Enter. When Enter was pressed, the script read those buffered rows instead of the actual gesture. The fix was to continuously discard incoming serial data while waiting for Enter, so the buffer is empty when recording starts.

4.4.4 Duplicate Repetition Numbers

Each new collection session restarted repetition numbering from zero, so the CSV files contained duplicate IDs. The fix was to make the collection script read the existing CSV and continue numbering from the last value. The training script was also updated to renumber repetitions based on row position, ignoring stored IDs if needed.

5. Discussion

The system could perform reliably for its intended purpose. The feature extraction approach with the Random Forest model was effective for three-class gesture classification with approximately 230 collected training samples. The gesture password and dodge game proved that the pipeline can support practical applications.

One limitation is that there are currently only 2 active gestures and 1 inactive one (idle). Adding more gestures would increase classification difficulty and require many more training data. The accelerometer is sensitive to sensor orientation because gravity contaminates raw readings. If the sensor is worn at a different tilt than during training, accuracy would likely be reduced.

Possible improvements include: removing gravity from accelerometer data using a high-pass filter or calibration, and adding more gesture classes with more training data.

6. Conclusions

A complete embedded gesture recognition system was built and demonstrated. The MSP430F5529 reads the MPU-6050 over I2C and sends data at 10 Hz over UART. A Python pipeline collects, extracts 42 features per sliding window, labels the window, and trains a Random Forest over three gesture labels. Real-time prediction drives a gesture password system and a gesture-controlled terminal game.

For the debugging process, 4 bugs were resolved: UART baud mismatch, I2C NACK handling, serial buffer issues, and repeated ID of new data.

The project serves as a stepping stone for the original motivation: a wrist-worn IMU system that can eventually recognize and evaluate badminton stroke technique, providing automated coaching feedback without a camera or a human observer.

Appendix A: Software File Descriptions

The following notes summarize the role of each software file used in the project.

data_storing.py

This script is run once per gesture type. It asks users to enter a label (e.g., “swipe_left”), then prompts users to press Enter before each repetition. We then perform the gesture, and the script records 20 rows of sensor data. This process is repeated as many times as possible (more data is better for model training). The output is a CSV file (e.g., swipe_left.csv) that grows and appends new training data whenever we perform new ones.

train.py

This script is run after collecting sufficient data for all gesture classes. It loads all CSV files, extracts features, and trains the Random Forest model. During execution, it prints the cross-validation accuracy and the top 10 most important features. The output is model.pkl, which stores the trained model.

predict1.py

This script is used to test live gesture recognition. It loads model.pkl, opens the serial port, and continuously prints the current predicted gesture and confidence score on a single line that updates in real time. The user performs gestures and observes the prediction change.

password.py

This script demonstrates the gesture-based password system. It prompts users to enter a sequence of four swipe gestures. The progress is displayed as each gesture is recognized (e.g., [RL__]). After four gestures, the user presses Enter to confirm, and the system prints either “ACCESS GRANTED” or “ACCESS DENIED” depending on whether the sequence matches the stored password.

game.py

This script runs a gesture-controlled dodge game in the terminal. Obstacles fall from the top of the screen, and the user swipes left or right to move the character. The game displays the board, the score, and the live gesture prediction. The speed increases over time.

inspect_data.py

This script is used to check the quality of the recorded data. It scans all CSV files and identifies rows that appear to be outliers. For each flagged row, it displays the values and allows the user to keep, delete, or replace them. This tool is used after data collection to clean the dataset before training.

main.c

This is the firmware that runs on the MSP430. It does three things: sets up the UART and I2C communication, wakes the MPU-6050 from sleep, and then enters an infinite loop where it reads the six sensor values from the MPU-6050 every 100 ms and sends them to the laptop as a line of comma-separated numbers over UART. This runs continuously as long as the MSP430 is powered.

Appendix B: Libraries Used

Library	**Purpose**
pyserial	Serial port communication
numpy	Numerical feature computation
pandas	CSV loading and manipulation in train.py
scikit-learn	RandomForestClassifier, cross_val_score
joblib	Saving/loading model.pkl
curses (stdlib)	Terminal game rendering in game.py
threading (stdlib)	Background gesture thread in game.py and password.py
select (stdlib)	Non-blocking stdin/serial polling in data_storing.py
collections.deque (stdlib)	Sliding window buffer in inference scripts

Table B1: Python libraries used.

References

[1] InvenSense. MPU-6000 and MPU-6050 Product Specification, Rev. 3.4. TDK InvenSense, 2013.

[2] Texas Instruments. MSP430F5529 LaunchPad User's Guide. Texas Instruments, 2013.

[3] Texas Instruments. MSP430x5xx and MSP430x6xx Family User's Guide (SLAU208). ti.com.

[4] Scikit-learn Developers. Random Forests -- scikit-learn documentation. scikit-learn.org. Accessed April 2026.

[5] Breiman, L. Random Forests. Machine Learning, vol. 45, no. 1, pp. 5-32, 2001.

[6] pySerial Developers. pySerial API Documentation. pyserial.readthedocs.io. Accessed April 2026.